UJIAN TENGAH SEMESTER

PRAKTIKUM KECERDASAN BUATAN

2026

Nama : Jelita Crisna Zalukhu

NIM : 241712022

Kom : A1

Uraian Penyelesaian dalam Pengoptimalisasian Model

Berikut merupakan uraian proses pengoptimalisasian model yang telah dilakukan. Penjelasan ini berfokus pada bagian-bagian kode yang mengalami pembaruan (update) guna mencapai performa model yang lebih stabil. Bagian yang tidak dicantumkan merupakan bagian standar yang dianggap telah memenuhi kriteria dasar pengembangan.

1. Mengganti fungsi `.unique()` menjadi `.value_counts()` pada kolom labels pada tahap awal penyiapan dataset

   Kode awal `(.unique())`, hanya memberikan daftar nama kelas yang ada (misal: ['cat', 'dog', 'wild']). Kode sesudah memakai `(.value_counts())` untuk memberikan jumlah total gambar untuk setiap kelas secara mendetail.

```
image_path = []
labels = []

for i in os.listdir("/content/animal-faces/afhq"):
  for label in os.listdir(f"/content/animal-faces/afhq/{i}"):
    for image in os.listdir(f"/content/animal-faces/afhq/{i}/{label}"):
      image_path.append(f"/content/animal-faces/afhq/{i}/{label}/{image}")
      labels.append(label)

data_df = pd.DataFrame(zip(image_path, labels), columns = ["image_path", "labels"])
print(data_df["labels"].value_counts())
data_df.head()
```

```
labels
cat     5653
dog     5239
wild    5238
Name: count, dtype: int64
```

   Alasan Perbaikan : Dalam *Deep Learning*, sangat penting untuk memastikan jumlah gambar antara Kucing, Anjing, dan Satwa Liar itu seimbang. Jika salah satu kelas (misal Anjing) jumlahnya jauh lebih sedikit, model akan kesulitan mengenali Anjing dan menjadi "bias" ke kelas yang datanya lebih banyak.

2. Mengganti metode `.sample()` manual dengan `train_test_split` dari library sklearn.

   Kode awal menggunakan `.sample(frac=0.7)` untuk melakukan pengambilan data secara acak murni. Ini membuat evaluasi model jadi tidak akurat. Setiap kali sel dijalankan, data yang masuk ke *train* dan *test* akan selalu berubah-ubah. Sulit untuk membandingkan hasil eksperimen jika datanya terus berganti.

   Oleh karena itu, dilakukan perbaikan dengan parameter `stratify=data_df["labels"],` untuk memastikan bahwa proporsi kelas di setiap bagian (train, val, test) tetap sama. Jika di data awal ada 33% Kucing, maka di folder train pun akan ada 33% Kucing. Lalu penggunaan `random_state=42` untuk memastikan bahwa pembagian data akan selalu sama setiap kali kode dijalankan.

```
from sklearn.model_selection import train_test_split
train, temp = train_test_split(
    data_df,
    test_size=0.3,
    stratify=data_df["labels"],
    random_state=42
)
val, test = train_test_split(
    temp,
    test_size=0.5,
    stratify=temp["labels"],
    random_state=42
)
print(train.shape, test.shape, val.shape)

(11291, 2) (2420, 2) (2419, 2)
```

3. Menambahkan teknik augmentasi acak pada data training dan menerapkan normalisasi pada seluruh data.

   Pada kode awal hanya mengubah ukuran gambar. Model hanya melihat gambar yang "begitu-begitu saja". Untuk itu agar model tidak overfitting, ditambahkan variasi seperti memutar gambar (Rotation), membalik secara horizontal (Flip), dan menggeser posisi (Translate).

   Dengan augmentasi, model belajar bahwa "Kucing" tetaplah "Kucing" meskipun posisinya miring atau menghadap ke kiri. Selanjutnya, ditambahkan fungsi Normalize dengan mean dan standar deviasi [0.5, 0.5, 0.5] untuk memastikan nilai piksel gambar berada dalam rentang yang serupa (biasanya -1 sampai 1).

   Hal ini sangat membantu proses optimasi model (Adam Optimizer) agar bekerja lebih stabil dan tidak mengalami lonjakan gradien yang drastis. Selain itu juga saya memisahkan fungsi transformasi antara data pelatihan dan data validasi/pengujian

```python
label_encoder = LabelEncoder()
label_encoder.fit(data_df["labels"])
```

```
LabelEncoder
LabelEncoder()
```

```python
train_transform = transforms.Compose([
    transforms.Resize((128,128)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)),
    transforms.ToTensor(),
    transforms.ConvertImageDtype(torch.float),
    transforms.Normalize([0.5,0.5,0.5],[0.5,0.5,0.5])
])

val_test_transform = transforms.Compose([
    transforms.Resize((128,128)),
    transforms.ToTensor(),
    transforms.ConvertImageDtype(torch.float),
    transforms.Normalize([0.5,0.5,0.5],[0.5,0.5,0.5])
])
```

4. Menghapus pemindahan data ke .to(device) di dalam Dataset Class.

   Pada kode awal label dan gambar dipindahkan ke GPU (device) langsung saat data baru dimuat di dalam kelas Dataset. Hal ini akan menyebabkan masalah jika memiliki ribuan gambar, memaksanya masuk ke GPU sekaligus di tahap ini akan membuat memori GPU penuh sebelum training dimulai.

   Jadi, saya membiarkan data tetap berada di CPU (RAM biasa) sebagai optimasi memori yang sangat penting agar program tidak mudah crash atau out of memory (OOM).

5. Membedakan objek transformasi antara data train dengan data test/val.

   Kode awal menggunakan satu fungsi transform yang sama untuk semua bagian data. Hal ini berisiko karena data pengujian (test) ikut terkena manipulasi acak yang tidak perlu. Sehingga saya memisahkan menjadi `train_transform` (untuk melatih model dengan variasi gambar) dan `val_test_transform` (untuk menguji model dengan gambar normal).

```python
train_dataset = CustomImageDataset(dataframe=train, label_encoder=label_encoder, transform=train_transform)
test_dataset = CustomImageDataset(dataframe=test, label_encoder=label_encoder, transform=val_test_transform)
val_dataset = CustomImageDataset(dataframe=val, label_encoder=label_encoder, transform=val_test_transform)
```

6. Menambahkan visualisasi data langsung dari objek train_dataset setelah terkena transformasi. Kode awal hanya menampilkan gambar asli secara acak dari folder (mentah). Ini hanya memperlihatkan isi dataset tanpa mengetahui apakah gambar tersebut sudah siap diproses model atau belum.

   Perubahan dilakukan dengan menambahkan visualisasi data yang diambil langsung dari objek `train_dataset`. Langkah ini sangat penting untuk memverifikasi secara visual apakah teknik *Data Augmentation* (seperti rotasi dan flip) serta normalisasi telah diterapkan dengan benar sebelum data masuk ke tahap pelatihan. Penggunaan fungsi `.permute(1,2,0)` juga ditambahkan untuk menyesuaikan urutan dimensi warna agar gambar dapat ditampilkan dengan tepat oleh library Matplotlib. Dengan melakukan pengecekan acak menggunakan `np.random.randint`, integritas antara gambar yang sudah diolah dan labelnya dapat dipastikan sudah sesuai, sehingga meminimalisir risiko kesalahan data yang dapat mengganggu akurasi model nantinya.

```
n_rows = 3
n_cols = 3

f, axarr = plt.subplots(n_rows, n_cols)

for row in range(n_rows):
  for col in range(n_cols):
    image = Image.open(data_df.sample(n=1)["image_path"].iloc[0]).convert("RGB")
    axarr[row, col].imshow(image)
    axarr[row, col].axis('off')

image, label = train_dataset[np.random.randint(0, len(train_dataset))]
plt.imshow(image.permute(1,2,0))
plt.axis('off')
```

7. Peningkatan jumlah iterasi pelatihan (*EPOCHS*) dari 10 menjadi 20.

   Peningkatan jumlah EPOCHS dilakukan untuk memberikan waktu lebih bagi model dalam mempelajari variasi data yang kini lebih beragam akibat penerapan *Data Augmentation*. Dengan jumlah iterasi yang lebih banyak, model dapat melakukan penyesuaian bobot secara lebih mendalam untuk mencapai titik konvergensi yang stabil. Dengan menambah durasi latihan namun tetap mempertahankan LR (Learning Rate) yang kecil, model dapat melakukan penyesuaian bobot secara lebih halus dan stabil. Langkah ini bertujuan untuk mencapai kondisi *Good Fit*, di mana akurasi pada data pelatihan dan data validasi dapat terus meningkat secara beriringan hingga mencapai hasil yang optimal.

8. Mengubah parameter `shuffle=True` menjadi `shuffle=False` pada val_loader dan test_loader.

   Perubahan dilakukan dengan menonaktifkan pengacakan data (`shuffle=False`) pada proses validasi dan pengujian. Hal ini bertujuan untuk menjaga urutan data tetap konsisten sehingga hasil evaluasi model dapat dianalisis dengan lebih mudah, terutama saat membandingkan prediksi model dengan label asli secara satu per satu.

   Sementara itu, fitur acak (`shuffle=True`) tetap dipertahankan pada data pelatihan agar model tidak menghafal urutan gambar dan mampu belajar secara lebih objektif. Dengan pengaturan ini, proses penilaian performa model menjadi lebih stabil dan terukur karena urutan data ujian tidak berubah-ubah di setiap pengujian.

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)
```

9. Penambahan lapisan Batch Normalization dan Dropout pada *CNN Architecture Improvement*

   Pembaruan pada kelas Net dilakukan dengan mengimplementasikan Batch Normalization (BatchNorm2d) di setiap blok konvolusi untuk menstabilkan distribusi aktivasi dan mempercepat proses *convergence* selama pelatihan. Selain itu, Dropout dengan *rate* 0.5 ditambahkan pada *fully connected layer* sebagai teknik regularisasi untuk memitigasi risiko overfitting.

   Penambahan Batch Normalization memungkinkan penggunaan *learning rate* yang lebih efektif tanpa risiko gradien yang tidak stabil, sementara Dropout memaksa model untuk mempelajari fitur yang lebih *robust* dan general. Kombinasi perubahan struktur ini secara signifikan meningkatkan performa model dari kondisi *underfitting* pada eksperimen awal menjadi kondisi Good Fit dengan akurasi yang tinggi dan stabil.

```python
class Net(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)

        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)

        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(128)

        self.pooling = nn.MaxPool2d(2, 2)
        self.relu = nn.ReLU()

        self.dropout = nn.Dropout(0.5)

        self.flatten = nn.Flatten()
        self.linear = nn.Linear(128 * 16 * 16, 128)
        self.output = nn.Linear(128, len(data_df['labels'].unique()))

    def forward(self, x):
        x = self.pooling(self.relu(self.bn1(self.conv1(x))))
        x = self.pooling(self.relu(self.bn2(self.conv2(x))))
        x = self.pooling(self.relu(self.bn3(self.conv3(x))))

        x = self.flatten(x)
        x = self.relu(self.linear(x))
        x = self.dropout(x)
        x = self.output(x)

        return x
```

10. Penggunaan mode `model.train()` dan `model.eval()`, serta perbaikan kalkulasi metrik.

    Pembaruan signifikan dilakukan dengan menggunakan perintah `model.train()` dan `model.eval()` di setiap awal fase training dan evaluasi. Langkah ini sangat krusial karena model kini menggunakan Batch Normalization dan Dropout, di mana kedua *layer* tersebut memiliki perilaku yang berbeda saat fase latihan (aktif) dan fase pengujian (non-aktif).

    Selain itu, proses pemindahan data ke GPU dilakukan secara tepat di dalam *loop* menggunakan `.to(device)`, yang menjamin efisiensi penggunaan memori. Perubahan pada perhitungan *average loss* dan *accuracy* juga membuat pelaporan statistik menjadi lebih standar dan akurat secara matematis. Secara keseluruhan, perbaikan logika ini memastikan bahwa model tidak hanya belajar dengan benar, tetapi juga dievaluasi secara adil tanpa gangguan dari mekanisme regularisasi selama pengujian. (Kode sulit dilampirkan karen terlalu panjang, untuk itu kode dapat dilihat pada file .ipynb)

11. Penerapan `model.eval()`, penggunaan `to(device)`, dan standardisasi perhitungan metrik.

    Selanjutnya ditambahkan fungsi `model.eval()` sebelum proses pengujian dimulai. Hal ini memastikan bahwa seluruh layer seperti *Batch Normalization* dan *Dropout* beralih ke mode Inference, sehingga hasil prediksi didasarkan pada bobot yang sudah stabil tanpa adanya manipulasi acak. Penggunaan `to(device)` pada *inputs* dan *labels* memastikan seluruh komputasi berjalan di atas hardware yang sama (GPU/CPU), menghindari error *runtime*. Selain itu, kalkulasi rata-rata *Loss* kini dibagi dengan jumlah total *batch* pada test_loader, yang menghasilkan nilai yang lebih representatif dibandingkan metode pembagian manual sebelumnya.

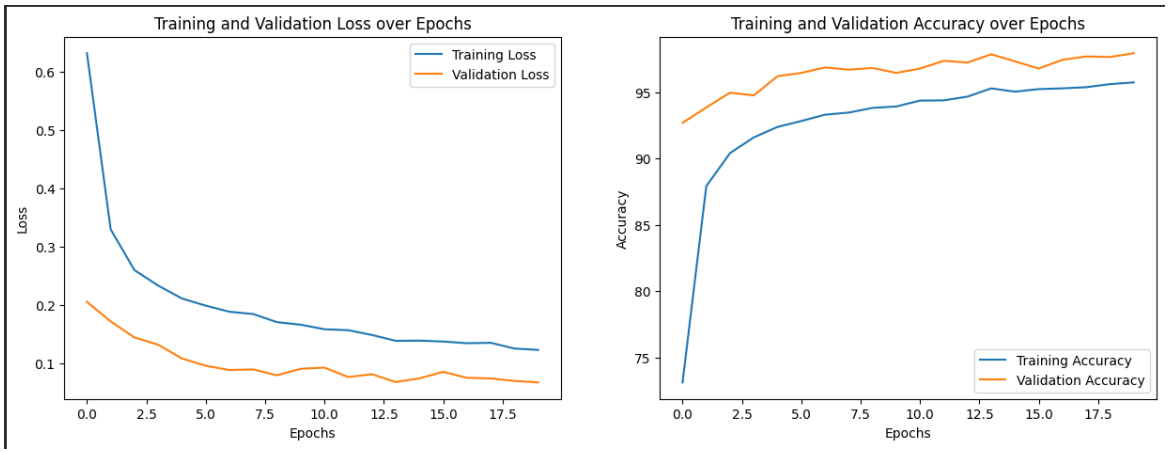
```
model.eval()
total_loss_test = 0
total_acc_test = 0

with torch.no_grad():
    for inputs, labels in test_loader:
        inputs, labels = inputs.to(device), labels.to(device)

        predictions = model(inputs)

        test_loss = criterion(predictions, labels)
        total_loss_test += test_loss.item()

        _, preds = torch.max(predictions, 1)
        total_acc_test += (preds == labels).sum().item()

avg_test_loss = total_loss_test / len(test_loader)
test_accuracy = (total_acc_test / len(test_dataset)) * 100
print(f"Test Accuracy: {test_accuracy:.2f}% | Test Loss: {avg_test_loss:.4f}")

Test Accuracy: 97.56% | Test Loss: 0.0742
```

12. Visualisasi performa model melalui grafik *Loss* dan *Accuracy*.

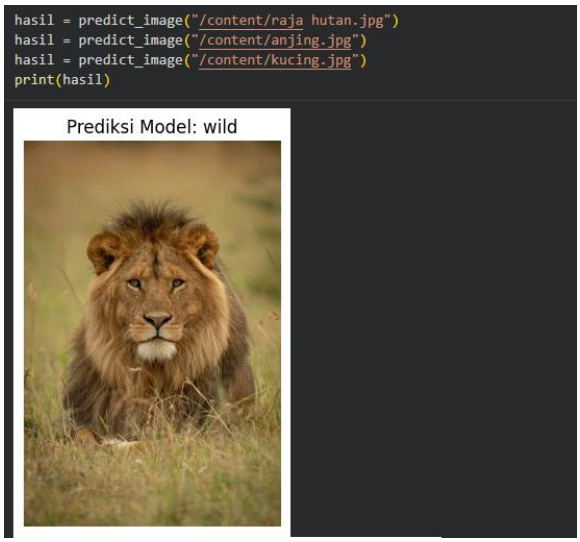
    Meskipun struktur kode visualisasi tidak mengalami perubahan, grafik yang dihasilkan menunjukkan perbedaan performa yang sangat signifikan antara model awal dan model setelah optimasi.

    Pada grafik *Loss*, terlihat tren penurunan yang stabil dan konvergen antara Training Loss dan Validation Loss, yang menandakan bahwa proses pembelajaran berjalan efisien. Sementara itu, grafik *Accuracy* menunjukkan bahwa kedua kurva bergerak naik secara berdampingan tanpa celah yang lebar, yang merupakan bukti kuat bahwa model telah mencapai kondisi Good Fit.

Hasil visualisasi ini memvalidasi bahwa penambahan teknik *Data Augmentation*, *Batch Normalization*, dan *Dropout* secara kolektif berhasil mengatasi masalah overfitting dan meningkatkan kemampuan generalisasi model dalam mengenali objek secara akurat.



13. Implementasi *error handling*, manajemen variabel global, dan integrasi visualisasi gambar. Pembaruan pada fungsi `predict_image` memberikan peningkatan signifikan dalam hal stabilitas dan kemudahan penggunaan. Penggunaan blok try-except dan validasi variabel global memastikan program tidak langsung berhenti (*crash*) jika file gambar tidak ditemukan atau jika komponen model belum siap. Secara teknis, penambahan `model.eval()` dan `torch.no_grad()` menjamin proses prediksi berjalan lebih efisien tanpa mengonsumsi memori tambahan untuk komputasi gradien.



(Berikut contoh hasil prediksi gambar. Kode lengkap dapat dilihat pada file)

Kesimpulan

Berdasarkan seluruh tahapan eksperimen, mulai dari persiapan data hingga pengujian akhir, dapat disimpulkan bahwa performa model mengalami peningkatan yang signifikan melalui serangkaian optimasi strategis. Berikut adalah poin-poin utama kesimpulannya:

1. Efektivitas Data Augmentation & Preprocessing.

   Penerapan teknik Data Augmentation (seperti *rotation, flip,* dan *affine*) terbukti efektif dalam memperkaya variasi dataset tanpa harus menambah data manual. Hal ini memperkuat kemampuan generalisasi model, sehingga model tidak hanya menghafal posisi objek, tetapi benar-benar mempelajari fitur visual dari setiap kategori hewan.

2. Peran Vital Batch Normalization & Dropout

   Perubahan arsitektur CNN dengan menambahkan Batch Normalization berhasil menstabilkan proses pelatihan dan mempercepat pencapaian titik convergence. Sementara itu, penggunaan Dropout berperan krusial dalam menekan angka overfitting, memastikan bahwa akurasi tinggi yang didapat pada data latihan juga konsisten saat diuji pada data validasi dan data pengujian (*test set*).

3. Pencapaian Kondisi Good Fit

   Melalui peningkatan jumlah Epochs menjadi 20 dan pengaturan Learning Rate yang tepat, model berhasil mencapai kondisi Good Fit. Hal ini ditunjukkan melalui *Learning Curves*, di mana grafik *Loss* menurun secara stabil dan grafik *Accuracy* meningkat secara beriringan antara data *Training* dan *Validation* tanpa adanya celah (*gap*) yang lebar.

4. Stabilitas Evaluasi dan Prediksi

   Penerapan logika model.eval() dan pemakaian DataLoader yang tidak diacak (*shuffle=False*) pada fase pengujian menjamin hasil evaluasi yang objektif dan konsisten. Fungsi prediksi akhir yang telah dilengkapi dengan *error handling* dan visualisasi langsung memberikan hasil yang akurat dan intuitif saat mengklasifikasikan gambar baru.

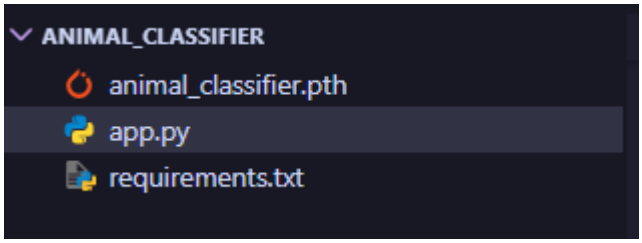
Pengaplikasian Model ke dalam Website

Website ini dibangun menggunakan Streamlit, sebuah framework berbasis Python yang memungkinkan pembuatan antarmuka website interaktif secara cepat tanpa harus menyusun kode HTML atau CSS dari nol, jadi tampilan yang dihasilkan juga tidak terlalu kompleks. Streamlit digunakan sebagai wadah untuk mengintegrasikan model AI ke dalam tampilan web yang fungsional, sehingga pengguna dapat mengunggah gambar dan melihat hasil prediksi secara langsung melalui browser. Berikut adalah langkah-langkahnya :

1. Menambahkan kode torch.save di akhir proses untuk mengekspor seluruh bobot dan parameter model yang sudah dilatih ke dalam file animal_classifier.pth, lalu mengunduhnya ke perangkat lokal agar "otak" model tersebut bisa digunakan kembali pada aplikasi website tanpa perlu melakukan pelatihan ulang.

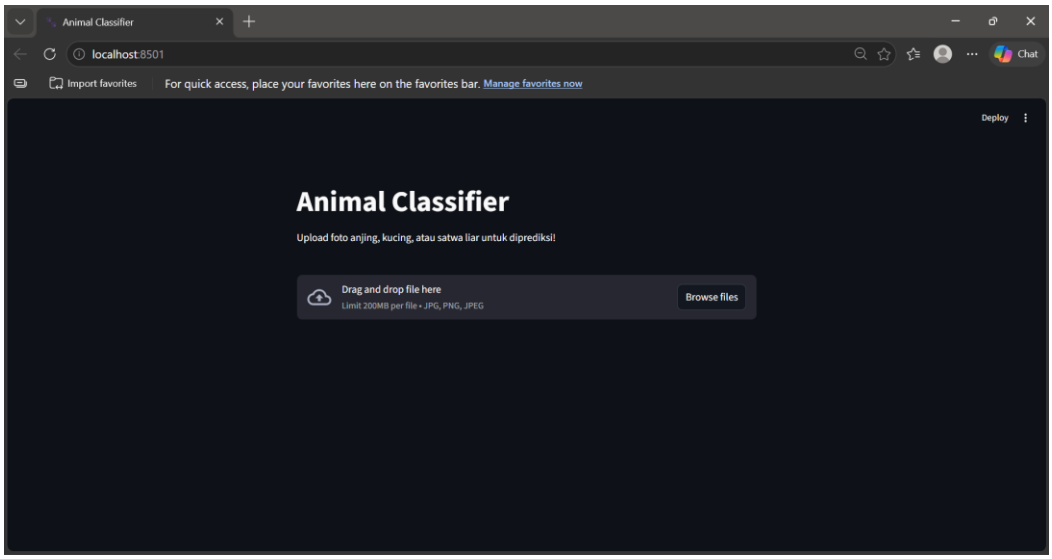
```
import torch

torch.save(model.state_dict(), 'animal_classifier.pth')

from google.colab import files
files.download('animal_classifier.pth')
```

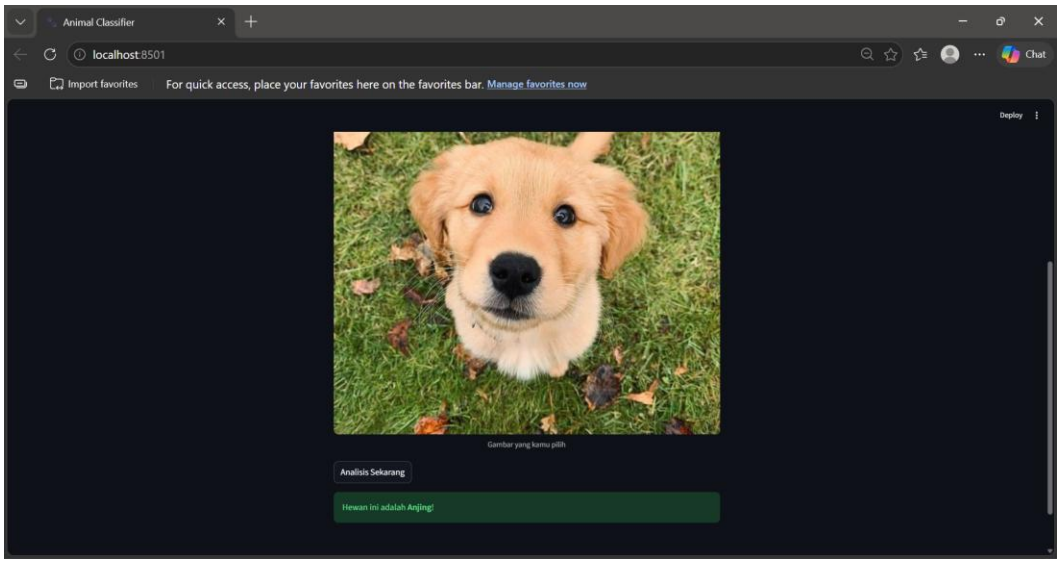
2. Tahap berikutnya adalah menyiapkan sebuah folder proyek yang berisi tiga file utama: app.py sebagai skrip inti untuk mengatur logika dan tampilan website, animal_classifier.pth yang merupakan file bobot model yang sudah diunduh, serta requirements.txt yang berisi daftar library seperti Torch dan Streamlit agar aplikasi dapat berjalan dengan stabil baik di komputer lokal maupun saat diunggah secara online.



3. Proses dilanjutkan dengan menginstal pustaka yang dibutuhkan melalui terminal menggunakan perintah `pip install torch torchvision pillow streamlit`, lalu menjalankan aplikasi dengan perintah streamlit `run app.py`. Langkah ini dilakukan di dalam folder proyek melalui VS Code atau CMD agar Streamlit bisa memproses kode dan membuka tampilan website secara otomatis di browser.

4. Setelah website terbuka di browser, pengguna akan disuguhkan antarmuka sederhana yang menyediakan fitur pengunggah gambar (*file uploader*). Pengguna hanya perlu memasukkan foto hewan yang ingin diidentifikasi, lalu menekan tombol ”Analisis Sekarang” untuk menjalankan model. Hasilnya, website akan menampilkan kembali gambar tersebut beserta teks keterangan klasifikasinya (Anjing, Kucing, atau Wild Animal) secara langsung di layar.





Link Repository:

https://github.com/jelitazalukhu/Responsi-Kecerdasan-Buatan